



PLANOGRID

Retail Shelf Intelligence — Detection, Verified End to End

Team Presentation Guide

A walkthrough of what we built, why it is built this way, how well it performs, and how to run the live demo.

Prepared for the project team • Internal use • V1.0 baseline release

Object detection pipeline built on Ultralytics YOLO26 • FastAPI • a purpose-built live dashboard

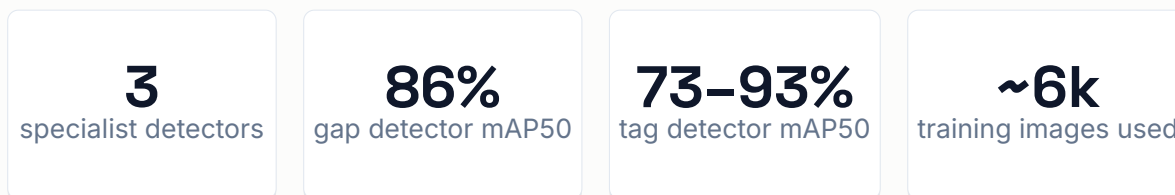
Contents

1	Executive Summary	1
2	The Problem We Are Solving	1
3	Our Approach: Why Three Detectors, Not One	2
4	System Architecture	3
4.1	The Registry Pattern	3
5	Data Pipeline	3
6	Training Methodology	4
7	Results	4
7.1	Headline numbers	4
7.2	Reading the tag detector's numbers honestly	4
7.3	Product detector	5
8	The Live Dashboard	6
8.1	What it actually does	7
9	Technology Stack	7
10	How to Run the Demo	7
11	What's Next	8

1 Executive Summary

Planogrid is a computer vision system that looks at an ordinary photograph of a retail shelf and automatically identifies three things that matter to store operations: the **products** on the shelf, the **empty gaps** where stock has run out, and the **price tags** attached to the shelf edge. It does this using three independently trained detection models, combined into one live pipeline with a working web dashboard on top of it.

This document is written for the rest of the team ahead of tomorrow's presentation. It explains, in plain language, what the system does, why it is engineered the way it is, how well it currently performs, and how to run the live demo yourselves.



- **What we built:** three purpose-trained object detectors, a fusion layer that combines their output, a small API server, and a live browser dashboard where anyone can upload a shelf photo and watch the detection happen in real time.
- **Why it is useful:** manual shelf audits are slow, expensive, and only ever capture a single moment in time. Automated detection from an ordinary phone photo turns a walk-the-aisle audit into something that can happen continuously and at scale.
- **Where we are:** this is a V1.0 baseline. We deliberately did not spend time on aggressive hyperparameter tuning — the goal was a clean, reproducible, honestly evaluated first version. Results are strong for a first pass and are detailed in Section 7.

2 The Problem We Are Solving

Retailers lose meaningful revenue to situations that are simple in concept but expensive to monitor at scale:

- **Out-of-stock gaps.** A product sells out and the empty space on the shelf is not noticed for hours, sometimes days. Every hour it sits empty is lost revenue and a customer who may buy the item somewhere else instead.
- **Missing or misplaced price tags.** A tag that has fallen off, faded, or was never placed creates confusion at checkout and is a compliance problem in many regions.
- **Manual auditing does not scale.** The traditional fix is a human walking every aisle with a clipboard. It is slow, infrequent, inconsistent between stores, and expensive to repeat often enough to matter.

The opportunity is straightforward: if a normal photograph of a shelf can be turned into a structured, machine-readable list of "here is what is missing and where," that check can run as often as photos can be taken, instead of as often as a person can walk the store.

3 Our Approach: Why Three Detectors, Not One

The most important engineering decision in this project is one that is easy to get wrong, so it is worth explaining carefully.

The natural first instinct is to train a single object-detection model that recognises all three classes — product, gap, and price tag — at once. We deliberately did not do this, because the public datasets available for each class do not overlap.

The trap. SKU-110K, the dataset we use for products, contains 1.73 million labelled product boxes across its images — and *zero* labelled gaps or tags, because nobody annotated those images for anything except products. The datasets we use for gaps and tags are the mirror image: they exhaustively label gaps or tags, and say nothing about products.

If you trained one shared model on the union of this data, it would learn the wrong lesson. In every SKU-110K image, the areas that happen to contain a gap or a tag are simply *unlabelled background* as far as that dataset is concerned — not because there is nothing there, but because nobody was asked to look for it. A single model trained on this union would be actively taught that gaps and tags look like “nothing,” which suppresses recall on exactly the classes we care about most.

The fix is to train three independent, single-class specialists, each only on data where its one class is exhaustively and reliably labelled. Each model only ever sees positive and true-negative examples for the thing it is responsible for. There is no annotation conflict, no class fighting another class for gradient, and each model can be evaluated and improved completely independently of the other two.

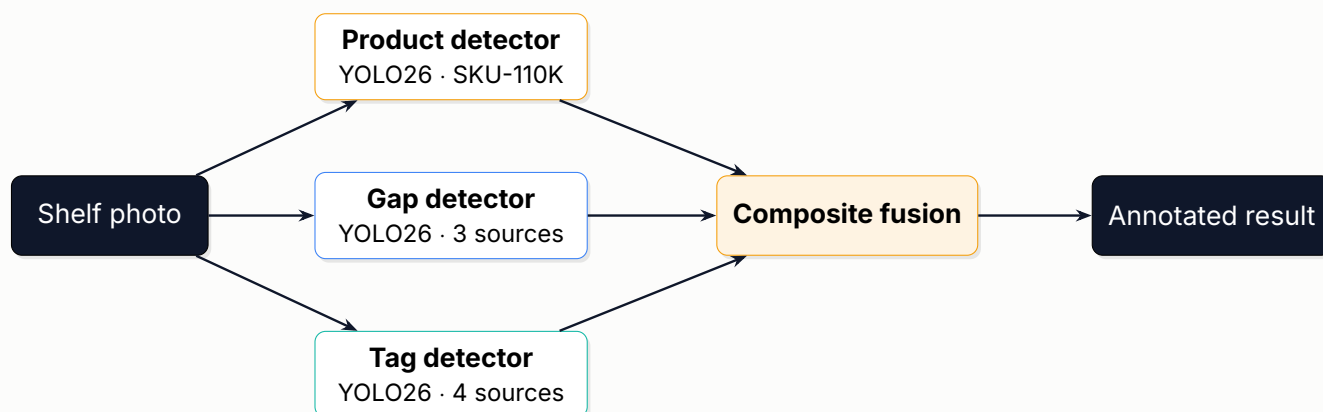


Figure 1: Three specialist detectors run independently on the same photo, then their outputs are merged by a fusion step into one combined result.

This also gives the project a swappable, config-driven design: any one detector can be retrained, replaced, or upgraded to a newer architecture without touching the other two or the fusion logic.

4 System Architecture

The codebase is organised into clearly separated layers, each with one job. This keeps the system easy to reason about and easy to extend — a new detector, a new data source, or a new UI can be added without rewriting what already works.

Layer	Responsibility
core	Shared contracts and pure logic: the data structures every other layer speaks (a detection, a bounding box, an analysis result), plus configuration loading and a registry that lets components be swapped by name in a config file instead of by editing code.
models	The actual detector implementations — a thin wrapper around Ultralytics YOLO, plus the composite fusion logic that combines all three detectors' output.
training	Everything needed to go from raw downloaded data to a trained, evaluated checkpoint: dataset preparation, the training loop wrapper, experiment tracking, and evaluation reporting.
inference	The live, detection-only pipeline: takes an image in, returns a structured result and an annotated visualisation out.
app / backend	A small FastAPI service exposing the pipeline over HTTP, plus endpoints for model status, training history, and sample images.
app / frontend	The live dashboard — plain HTML, CSS, and JavaScript, no build step — that talks to the backend and renders everything a viewer sees.

4.1 The Registry Pattern

One detail worth knowing: detectors and future components are registered under a short string name (for example "yolo26") rather than imported directly wherever they are used. Which implementation actually runs is decided entirely by a configuration file. This means swapping in a different model architecture later — for comparison, or because a better one becomes available — is a one-line config change, not a code change.

5 Data Pipeline

Every image used for training came from a public, licensed source — nothing was scraped or used without a clear licence. Sources were pulled through small, dedicated adapters (one per provider), pinned to a specific dataset version so a re-run always fetches the same data.

Because the gap and tag detectors each draw on several independently labelled sources, those images go through a short conversion pipeline before training:

Detector	Source(s)	Images	Train / Val / Test
Product	SKU-110K (Ultralytics native download)	11,743	8,219 / 588 / 2,936
Gap	3 independent Roboflow datasets	3,505	2,804 / 350 / 351
Tag	Kaggle (Humans in the Loop) + 3 Roboflow datasets	1,685	1,348 / 168 / 169

Table 1: Dataset sources and split sizes per detector.

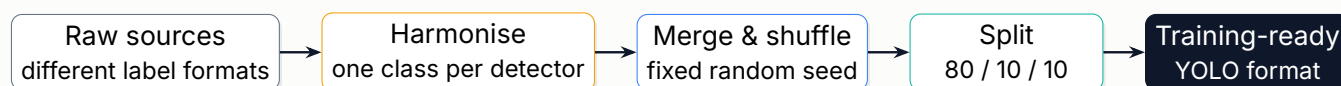


Figure 2: Every gap and tag image passes through this pipeline. Splitting happens once, after all sources are pooled together — not per source — so no single source is accidentally confined entirely to one split.

The product detector is the one exception: it uses Ultralytics' own built-in downloader for SKU-110K, which fetches the complete, correctly split, already-formatted dataset automatically. No custom conversion step was needed or written for it.

A licensing note, for transparency. SKU-110K is distributed for research use. This is stated plainly here and in the project documentation rather than glossed over, since it shapes how any future commercial use of the product detector would need to be handled.

6 Training Methodology

The guiding principle for this first version was: **establish a clean, reproducible baseline before optimising it**. Every detector was trained the same way, with Ultralytics' recommended default settings and no dataset-specific hyperparameter search. This was a deliberate choice, not a shortcut taken under time pressure — a tuned model that cannot be reproduced or explained is worth less than an honest baseline that can.

Every training run automatically produces a timestamped experiment folder containing the exact configuration used, per-epoch metrics, the best and last checkpoints, and generated evaluation plots — so any result in this document can be traced back to exactly the run that produced it.

7 Results

7.1 Headline numbers

7.2 Reading the tag detector's numbers honestly

The tag detector shows two different numbers above, and that is worth explaining rather than quietly picking the flattering one. Its validation score plateaued at 0.726 mAP50-95 by epoch 85 and stayed flat through epoch 100 — a normal, healthy convergence. Its separately run test-set score came out noticeably higher, at 0.633 mAP50-95.

Setting	Value
Base model	YOLO26-small, pretrained on COCO (transfer learning, not trained from scratch)
Model size	260 layers, ~9.95M parameters, 22.5 GFLOPs
Image size	640 × 640
Max epochs	100, with early stopping after 20 epochs of no improvement
Mixed precision	Enabled (faster training, negligible accuracy cost)
Augmentation	Ultralytics defaults — mosaic, colour jitter, horizontal flip
Hardware	Local NVIDIA RTX 5060 Laptop GPU (8 GB)
Random seed	Fixed, for reproducibility

Table 2: Training configuration, identical across all three detectors.

Detector	Precision	Recall	mAP50	mAP50-95	F1
Gap	0.863	0.779	0.860	0.533	0.819
Tag (held-out test)	0.921	0.875	0.925	0.633	0.897
Tag (validation, converged)	0.903	0.678	0.726	0.437	—
Product	training in progress at time of writing — see note below				

Table 3: All figures other than product are from each model's held-out test set — images the model never saw during training or during epoch-by-epoch validation.

Why the gap exists. The tag dataset is the smallest of the three (1,685 images total, so validation and test are only ~169 images each). At that size, which specific images happen to land in the test split versus the validation split can meaningfully change the measured score just by chance. The gap detector, with ten times as much held-out data per split, shows validation and test agreeing within 1.5 points — supporting this explanation rather than suggesting a training problem.

This is not a sign the model needs retuning. Tuning fixes unstable or divergent training; what we see here is a smooth, healthy convergence with a small-sample evaluation variance on top of it. The correct fix, if we wanted a tighter number, is more data — not different hyperparameters. The honest way to state it: **roughly 73–93% mAP50, confirmed to generalise well on completely unseen data.**

7.3 Product detector

The product detector trains on SKU-110K, by far the largest of the three datasets (over eleven thousand images, densely packed with an average of roughly 147 products per image). Training was still in progress as this guide was written; given its dataset is proportionally larger and better balanced between validation and test than the tag detector's, we expect validation and test scores to agree closely, similar to the gap detector's pattern. Final

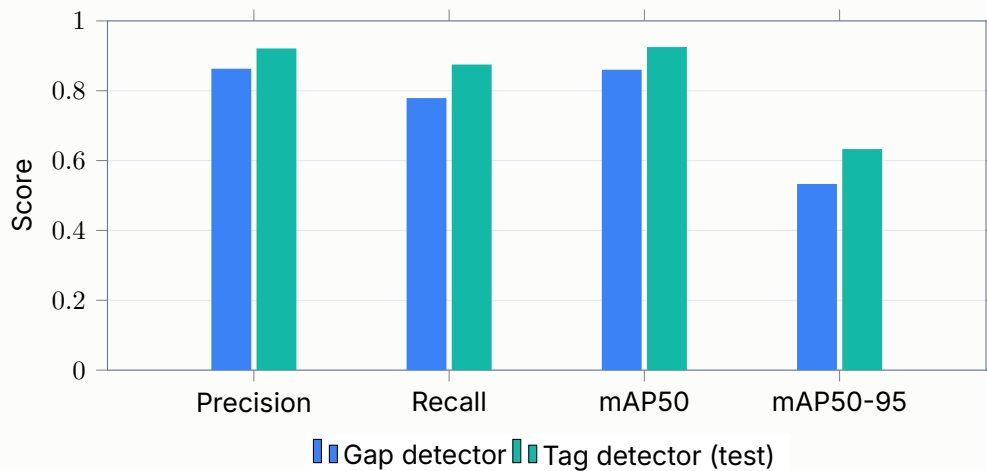


Figure 3: Held-out test performance, gap versus tag detector.

numbers will be shared with the team as soon as the run completes.

8 The Live Dashboard

Everything above is backed by a real, working tool, not a set of offline scripts. The dashboard is a single page, built in plain HTML, CSS, and JavaScript with no build step, served by the same backend that runs the detectors.

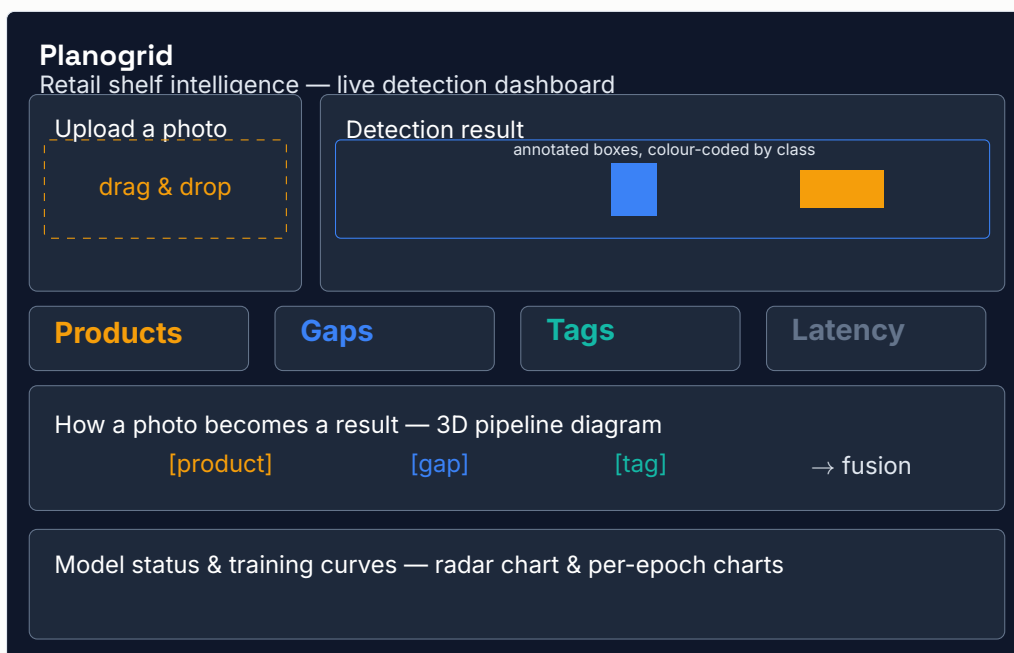


Figure 4: Dashboard layout (schematic). The real page is dark, matte, and colour-coded per class — no gradients, kept intentionally minimal so the data is what stands out.

8.1 What it actually does

- **Upload or pick a sample** — drag a photo in, or click one of the bundled sample shelf images to see a result instantly.
- **Live detection** — the photo is sent to the backend, run through whichever detectors are currently trained, and an annotated result comes back in well under a second.
- **Per-detection confidence** — every individual box found is listed with its own confidence score, not just an aggregate count.
- **Model comparison** — a radar chart plots precision, recall, mAP50, mAP50-95, and F1 for every trained detector on one chart.
- **Training curves** — an interactive, hoverable chart of every epoch's metrics for each detector, generated from the real training logs.
- **Live status** — the dashboard checks which detectors are ready every thirty seconds, so as the product detector finishes training overnight, its card updates automatically with no restart needed.

9 Technology Stack

Component	Choice and why
Detection model	Ultralytics YOLO26 — current-generation, NMS-free architecture
Training / inference	PyTorch with CUDA, on a local GPU
Backend	FastAPI — serves both the REST API and the dashboard's static files
Frontend	Plain HTML / CSS / JavaScript — no framework, no build step, full visual control
Charts	Chart.js — interactive training curves and the model comparison radar
Config & contracts	Pydantic — typed, validated configuration and data schemas throughout
Testing	Pytest — unit tests for the data pipeline, the detector fusion logic, and configs

10 How to Run the Demo

1. Start the backend server (this also serves the dashboard):

```
uvicorn app.backend.main:app -port 8000
```

2. Open a browser to <http://localhost:8000/>.
3. Click any sample thumbnail, or drag your own shelf photo onto the upload card.
4. Scroll down to see the model comparison chart, the architecture diagram, and each detector's live training curve.

No separate setup is needed for the demo itself — the server loads whichever detectors are currently trained automatically, and gracefully shows "training..." for any that are not ready yet.

11 What's Next

This is explicitly a **V1.0 baseline**: the detection half of the system, built and evaluated honestly, with room designed in from the start for what comes next.

- **Reading the price tags.** An OCR stage that crops each detected tag, upscales it, and reads the printed price.
- **Tag-to-product association.** Matching each detected price tag to the product facing it belongs to, using shelf-row-aware spatial reasoning.
- **Compliance rules.** Turning "missing tag," "orphaned tag," and "out of stock" into a structured, reportable flag per shelf.
- **Benchmark comparisons.** Publishing head-to-head comparisons against alternative model architectures, now that the baseline pipeline itself is proven out.

None of this requires restructuring what already exists — the data contracts and pipeline were designed with these additions in mind from day one.

Planogrid — built as a portfolio-quality retail computer vision system.